System Design Document – WSSOA (IT644)
Autumn 2026

Project Title :

PriceWise – Compare, Analyze, Shop Smarter

A Web-Based AI-Powered Smart Shopping Assistant

Date of Submission: **29/08/2026**

Submitted By:

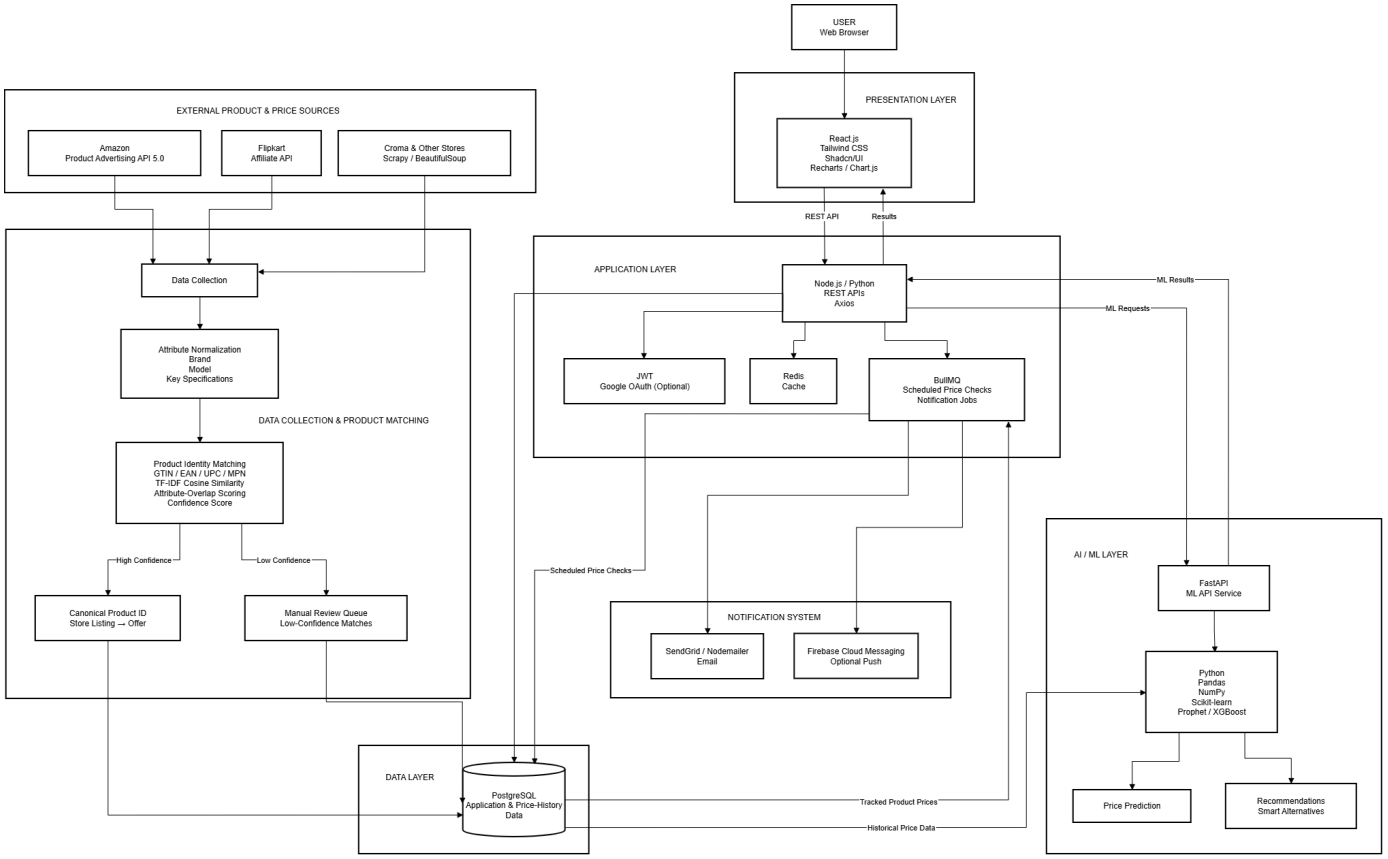
Sr. No.	Student Name	Student ID
1	Diya Jain	202512014
2	Piyush Tanwani	202512021
3	Divyesh Dandwani	202512043
4	Harshil Dodwani	202512044
5	Vivek Dhanwani	202512045
6	Yash Gangwani	202512048
7	Aman Rohera	202512049
8	Priya Shah	202512100

Submitted To:

Prof. Jayprakash Lalchandani
Course Instructor - WSSOA (IT644)
DAU, Autumn 2026

Architecture diagram

PriceWise follows a layered web application architecture that separates the user interface, application logic, data management, AI/ML processing, and external data collection. The architecture enables the system to collect product and price data from multiple sources, process and store it, and provide price comparison, prediction, recommendation, and alert features through the web application.



Drive link For Above Diagram: Architecture Diagram.png

Architecture Description

The PriceWise architecture consists of the following major components:

1. **Presentation Layer:**
   The frontend is developed using React.js with Tailwind CSS and Shadcn/UI. Recharts/Chart.js is used to display price history, trends, and analytical information to users.

2. **Application Layer:**
   The backend uses Node.js/Python and provides REST APIs for product search, comparison, price analysis, user management, watchlists, recommendations, and notifications. JWT and optional Google OAuth are used for authentication. Redis provides caching, while BullMQ manages scheduled price checks and notification jobs

   .

3. **Data Layer:**
   PostgreSQL stores application data, product information, offers, user data, watchlists, notifications, and historical price information.

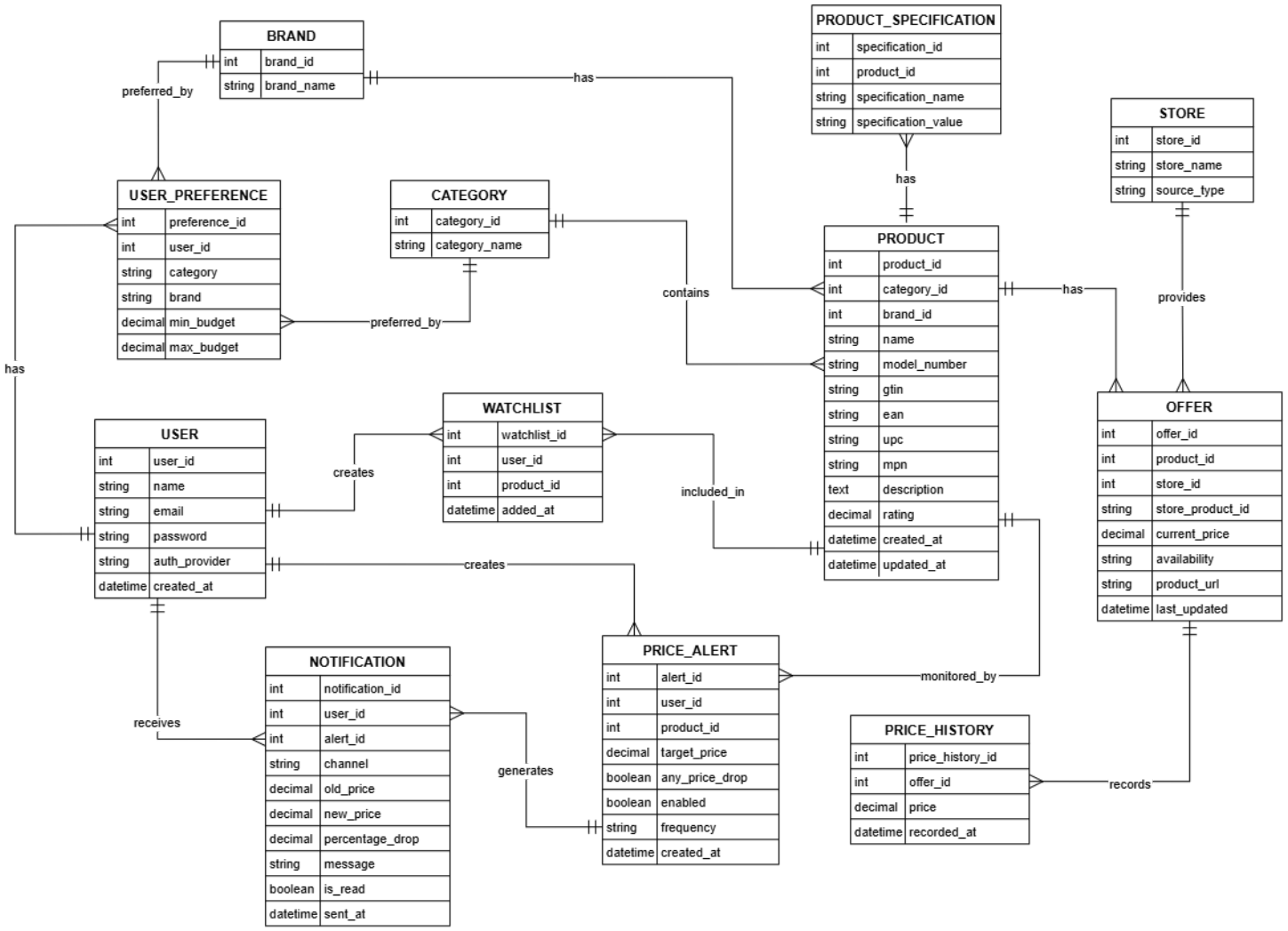
4. **AI/ML Layer:**
   Python-based AI/ML components use Pandas, NumPy, Scikit-learn, and Prophet/XGBoost for price prediction and recommendation-related processing. FastAPI provides the ML API service through which the backend communicates with the ML functionality.

5. **Data Collection and Product Matching:**
   Product and price data are collected from Amazon, Flipkart, Croma, and other available sources. The data is normalized and processed through product identity matching using identifiers such as GTIN/EAN/UPC/MPN, TF-IDF similarity, attribute-overlap scoring, and confidence scoring. High-confidence matches are assigned a Canonical Product ID, while low-confidence matches can be sent for manual review.

6. **Notification System:**
   BullMQ performs scheduled price checks for tracked products. When an applicable price change occurs, notifications can be delivered through SendGrid/Nodemailer, with Firebase Cloud Messaging available as an optional push-notification channel.

Database Schema (ER Diagram)

The PriceWise database schema is designed using PostgreSQL to store and manage user, product, store, offer, pricing, watchlist, alert, and notification data. The schema supports the core functionality of PriceWise, including cross-store price comparison, historical price tracking, personalized shopping, price-drop alerts, and AI/ML-based price analysis.



Drive link For Above Diagram: ER Diagram.jpeg

ER Diagram Description

- The User entity stores account information and is associated with user preferences, watchlists, price alerts, and notifications. `USER_PREFERENCE` stores information such as preferred categories, brands, and budget ranges to support personalized shopping recommendations.

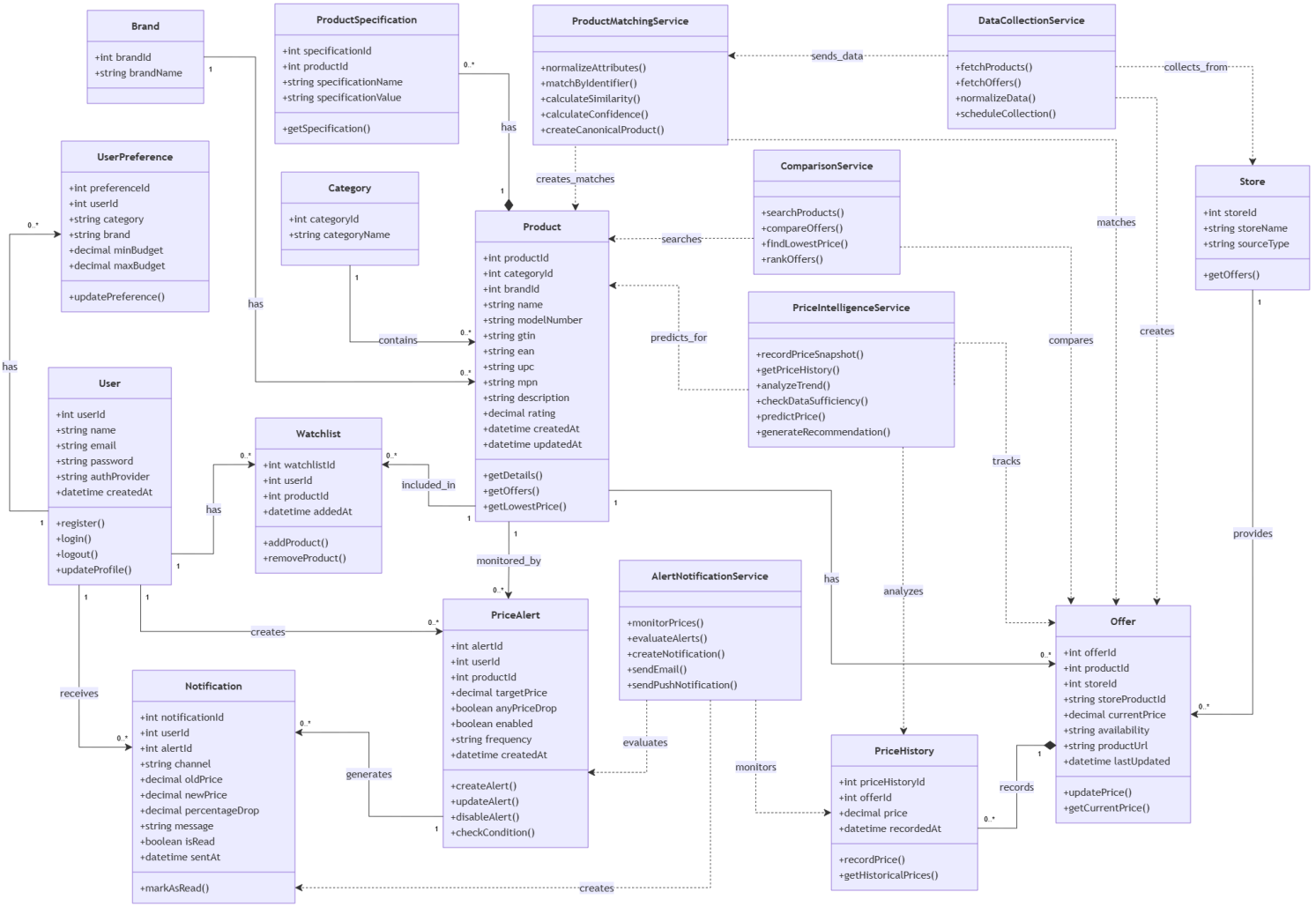
- The Product entity represents the canonical PriceWise product. Products are associated with a Category and Brand, while `PRODUCT_SPECIFICATION` stores individual specifications such as RAM, storage, size, or other product-specific attributes required for product comparison.

- The Store entity represents external shopping sources such as Amazon, Flipkart, Croma, and other available stores. Each Offer represents a store-specific listing for a canonical product and contains information such as the current price, availability, store product identifier, product URL, and last update time.

- The Price History entity stores timestamped price records for each offer. This historical data supports price-trend visualization, price analysis, and the historical data required by the PriceWise price-prediction functionality.

- The Watchlist entity connects users with products they want to monitor. The Price Alert entity stores target-price or price-drop monitoring preferences. When an applicable price change occurs, the system can generate a Notification, which records the notification channel and relevant price-change information.

Class Diagram



Drive link For Above Diagram: Class Diagram.png

Class Diagram Description

- **Core Classes:**
  The diagram contains the main entities of PriceWise such as User, Product, Store, Offer, PriceHistory, Watchlist, PriceAlert, and Notification.
- **Product & Price Management:**
  A Product can have multiple Offers from different Stores, and each offer maintains its PriceHistory for tracking price changes over time.
- **User Features:**
  Users can save products in their Watchlist, create PriceAlerts, and receive Notifications when alert conditions are met.
- **Service Layer:**
  Service classes such as DataCollectionService, ProductMatchingService, ComparisonService, PriceIntelligenceService, and AlertNotificationService handle the main business logic of the system.
- **Overall Flow:**
  The diagram shows how PriceWise collects product data, matches products across stores, compares prices, analyzes historical data for predictions/recommendations, and sends price-drop notifications to users.

REST API Contracts

- **REST API Overview**

The REST API specification defines the proposed communication interface between the PriceWise frontend and backend. It specifies the API endpoints, HTTP methods, parameters, request and response structures, authentication requirements, and HTTP status codes. The Swagger/OpenAPI specification serves as the API contract for backend development and frontend-backend integration.

- **API Modules**

No.	API Module	Purpose
1	Authentication	User registration and login
2	User Management	User profile and preferences
3	Product	Product information and details
4	Product Search	Search and filter products
5	Product Comparison	Compare products and offers
6	Price History	Retrieve historical price data
7	Watchlist	Manage watched products
8	Price Alerts	Create and manage price alerts
9	Notifications	Retrieve user notifications
10	Recommendations	Personalized product recommendations
11	Price Prediction	Future price prediction

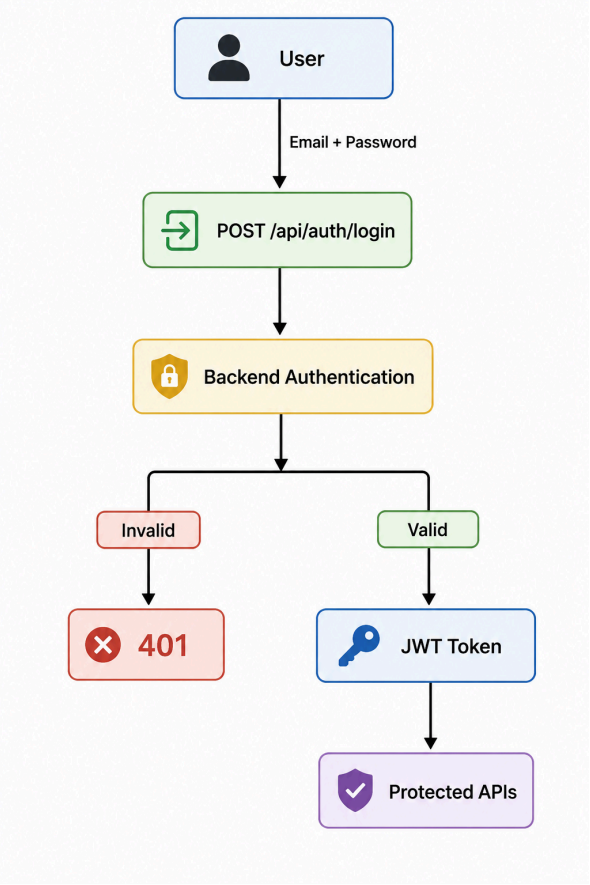
- **API Endpoint Specification**
  The following table defines the proposed REST API contract for PriceWise.

Module	Method	Endpoint	Purpose	Authentication
Authentication	POST	/api/auth/register	Register a new user	No
Authentication	POST	/api/auth/login	Authenticate user	No
User	GET	/api/users/me	Get current user details	JWT
User	GET	/api/users/me/preferences	Get user preferences	JWT
User	PUT	/api/users/me/preferences	Update user preferences	JWT
Products	GET	/api/products	Retrieve products	No
Products	GET	/api/products/{id}	Get product details	No
Search	GET	/api/products/search	Search and filter products	No
Comparison	GET	/api/products/compare	Compare products and offers	No
Price History	GET	/api/products/{id}/price-history	Retrieve historical prices	No
Watchlist	GET	/api/watchlist	Get user's watchlist	JWT
Watchlist	POST	/api/watchlist	Add product to watchlist	JWT
Watchlist	DELETE	/api/watchlist/{productId}	Remove product from watchlist	JWT
Price Alert	GET	/api/alerts	Get user's alerts	JWT
Price Alert	POST	/api/alerts	Create price alert	JWT
Price Alert	PUT	/api/alerts/{id}	Update price alert	JWT
Price Alert	DELETE	/api/alerts/{id}	Delete price alert	JWT

Notifications	GET	/api/notifications	Get user notifications	JWT
Recommendations	GET	/api/recommendations	Get product recommendations	JWT
Prediction	GET	/api/products/{id}/prediction	Get price prediction	No

- **Authentication**
  - PriceWise uses JWT-based authentication for protected API endpoints. After successful authentication, the client receives a JWT token and sends the token with subsequent protected requests.
  - Protected requests use the following HTTP header:
    Authorization: Bearer <JWT_TOKEN>
  - The JWT mechanism is proposed for endpoints requiring authenticated access such as user preferences, watchlist, price alerts, notifications, and recommendations.

- **Authentication Flow**



- **Request and Response Formats**
  - Request Format

    API requests use standard HTTP methods. JSON is used as the primary data format for request bodies and responses. Query parameters are used for search, filtering, sorting, and other optional parameters.

    Example:

    POST `/api/auth/login`
    Content-Type: application/json

```
{
 "email": "user@example.com",
 "password": "password"
}
```

  - Response Format

    Successful API requests return structured JSON responses containing the requested resource or operation result.

    Example:

```
{
  "message": "Request successful",
  "data": {}
}
```

- **API Request and Response Examples**

  You do not need to show request/response examples for every endpoint. The following four important APIs are sufficient for the design document.

Login API

Endpoint

POST `/api/auth/login`

Request

```
{
  "email": "user@example.com",
  "password": "password"
}
```

Response

```
{
  "message": "Login successful",
  "token": "<JWT_TOKEN>",
  "user": {
    "user_id": 1,
    "name": "User",
    "email": "user@example.com"
  }
}
```

Description

The login endpoint authenticates the user and returns a JWT token that can be used for accessing protected API endpoints.

Product Details API

Endpoint

```
GET /api/products/{id}
```

Example Response

```
{
  "product_id": 101,
  "name": "Example Product",
  "brand": "Example Brand",
  "model_number": "ABC123",
  "rating": 4.5,
  "offers": [
    {
      "store_name": "Amazon",
      "current_price": 79999,
      "availability": "Available"
    },
    {
      "store_name": "Flipkart",
      "current_price": 78500,
      "availability": "Available"
    }
  ]
}
```

This example follows the structure provided in the existing API design.

Price History API

Endpoint

```
GET /api/products/{id}/price-history
```

Example Response

```
{
  "product_id": 101,
  "price_history": [
    {
      "price": 82000,
      "recorded_at": "2026-08-01"
    },
    {
      "price": 79999,
      "recorded_at": "2026-08-15"
    }
  ]
}
```

This follows the proposed example structure in the supplied API design.

Price Prediction API

Endpoint

```
GET /api/products/{id}/prediction
```

Example Response

```
{
  "product_id": 101,
  "prediction": {
    "predicted_price": 75999,
    "horizon": "30_days",
    "trend": "downward",
    "confidence": "moderate"
  },
  "disclaimer": "Predicted prices are estimates and do not guarantee
future prices."
}
```

Description

The price prediction endpoint provides the proposed interface for retrieving an estimated future product price, prediction horizon, trend, and confidence information.

- ## API Data Models

  This subsection is useful because Swagger/OpenAPI needs schemas for request and response objects.

  1 User

  ```
  User
  ├── user_id
  ├── name
  ├── email
  └── role
  ```

  2 Product

  ```
  Product
  ├── product_id
  ├── name
  ├── brand
  ├── category
  ├── model_number
  ├── description
  ├── specifications
  └── rating
  ```

  3 Offer

  ```
  Offer
  ├── offer_id
  ├── product_id
  ├── store_name
  ├── current_price
  ├── availability
  └── last_updated
  ```

  4 Price History

  ```
  PriceHistory
  ├── product_id
  ├── store
  ├── price
  └── recorded_at
  ```

5 Prediction

```
Prediction
├── product_id
├── predicted_price
├── horizon
├── trend
└── confidence
```

6 Recommendation

```
Recommendation
├── product_id
├── recommendation
└── factors
```

7 Watchlist

```
Watchlist
├── watchlist_id
├── user_id
├── product_id
└── created_at
```

8 Price Alert

```
PriceAlert
├── alert_id
├── user_id
├── product_id
├── target_price
├── status
└── frequency
```

- **HTTP Status Codes**

  These status codes are included in the existing API design.

Status Code	Meaning
200	Request successful
201	Resource successfully created
400	Invalid request
401	Authentication required or failed
403	Access denied
404	Resource not found
409	Resource conflict
500	Internal server error

- **Error Handling**

  PriceWise will use standardized error responses for API failures. Error messages will provide meaningful information to API consumers while avoiding exposure of sensitive information or internal implementation details.

  Example

```
{
  "error": {
    "code": "PRODUCT_NOT_FOUND",
    "message": "The requested product was not found."
  }
}
```

- **Swagger / OpenAPI Specification**

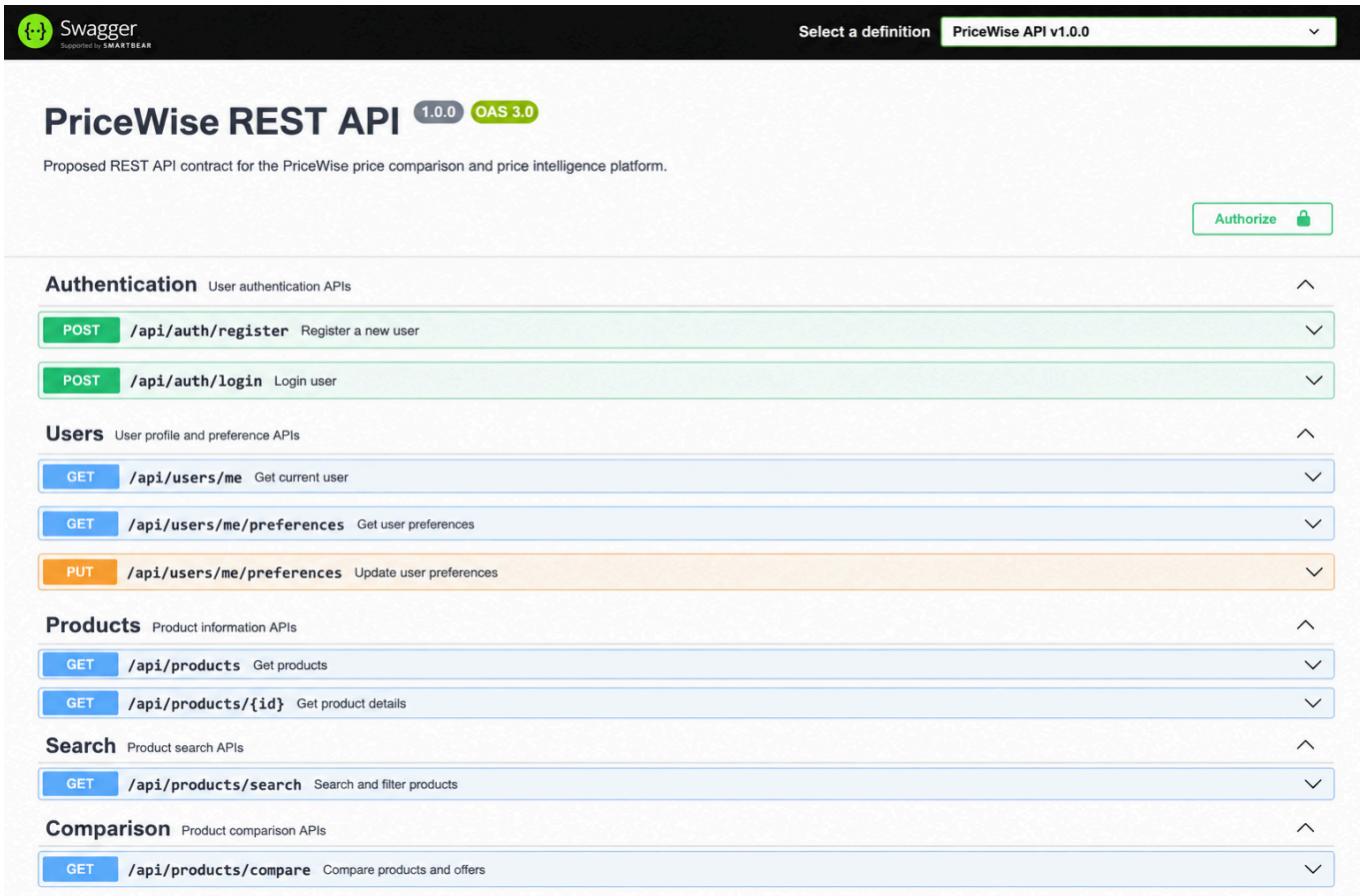
1 OpenAPI Specification

- The proposed PriceWise REST APIs will be documented using the OpenAPI specification. The specification defines the available endpoints, HTTP methods, parameters, request bodies, response schemas, authentication requirements, and HTTP status codes.

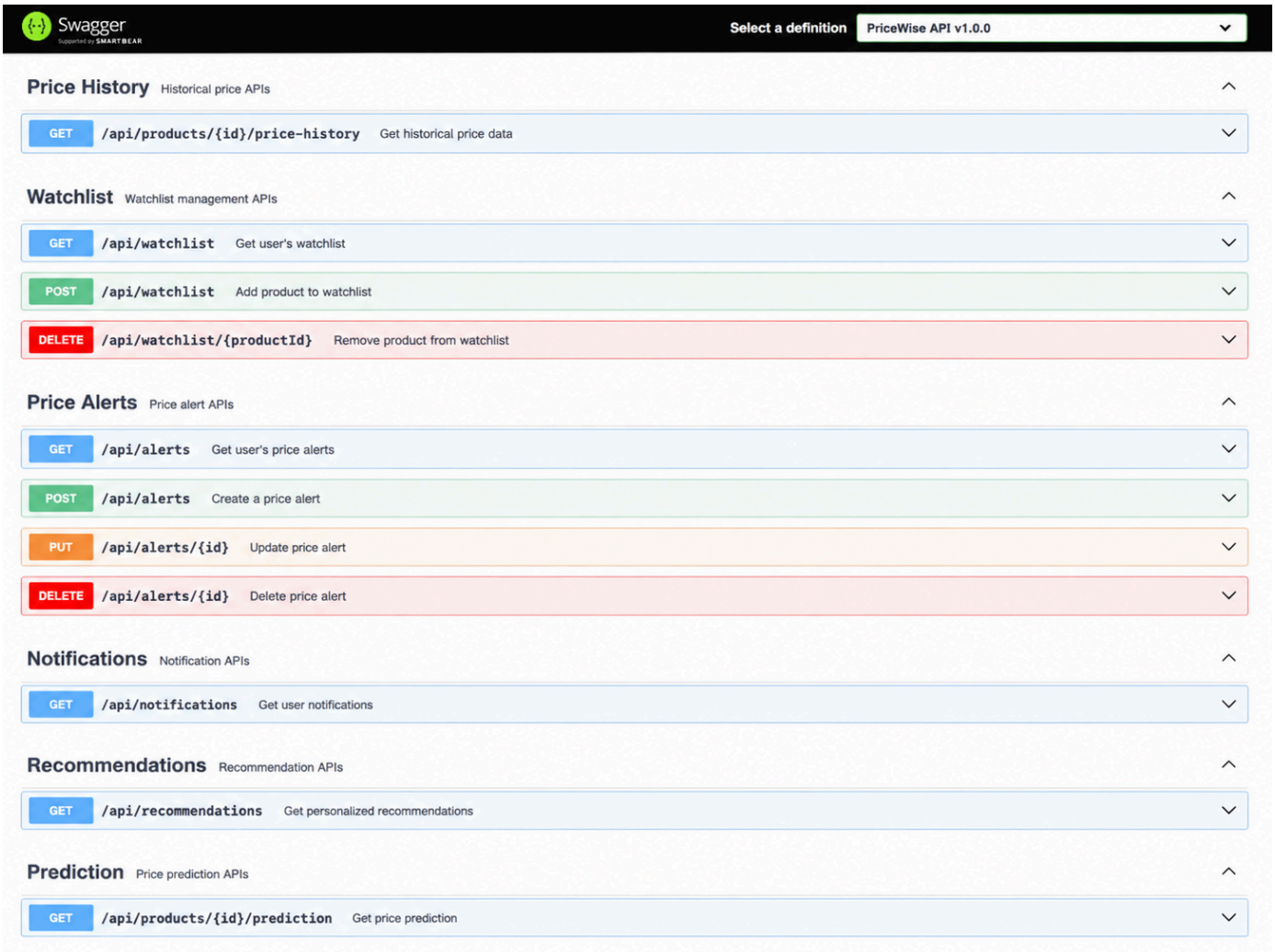
2 Swagger UI

- Swagger UI will provide an interactive representation of the OpenAPI API contract. It will allow developers and testers to view the proposed endpoints, inspect request and response structures, and test the APIs after backend implementation.

- **Swagger/OpenAPI Screenshots**



This screenshot presents the Swagger/OpenAPI specification for the core PriceWise APIs, including user authentication, user profile and preference management, product information, product search, and product comparison operations.



This screenshot presents the Swagger/OpenAPI specification for PriceWise price intelligence and user-specific services, including historical price retrieval, watchlist management, price alerts, notifications, personalized recommendations, and price prediction.

Wireframes - UI/UX mockup

This section documents each screen of the PriceWise platform. For every screen, we present:
1. Screen Purpose & User Goals
2. Low-Fidelity Blueprint Wireframe (Structural Layout with Image & Field Placeholders)
3. High-Fidelity UI/UX Figma Mockup (Final Production Visual Design)
4. Component & Layout Breakdown Table
5. Screen-Specific Acceptance Criteria

Landing Page & Smart Home Dashboard

Route: / (Home)

Screen Purpose & User Goal: The primary entry point of the platform designed to communicate the core value proposition, facilitate instant multi-store product searches, showcase live price monitoring preview widgets, and present macro platform trust metrics.

A. Low-Fidelity Blueprint Wireframe (Structural Layout & Placeholders):

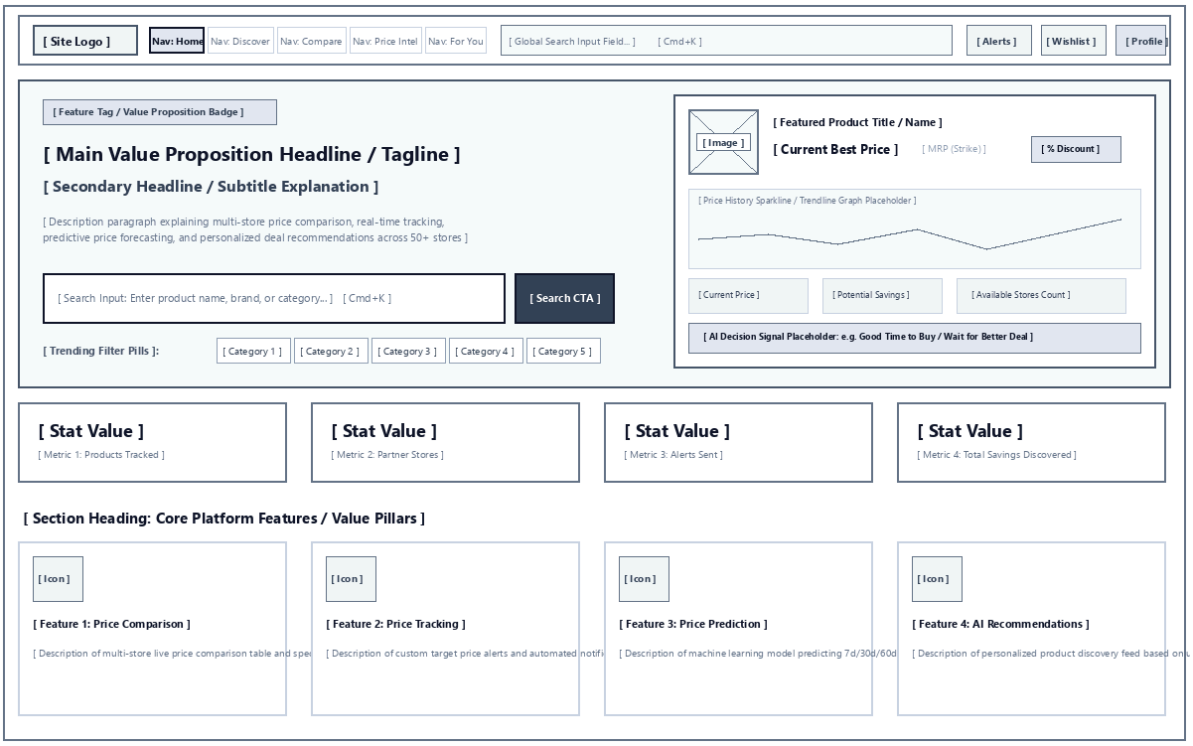


Figure 6.1(a): Low-Fidelity Blueprint Wireframe — Landing Page & Smart Home Dashboard

B. High-Fidelity UI/UX Figma Mockup :

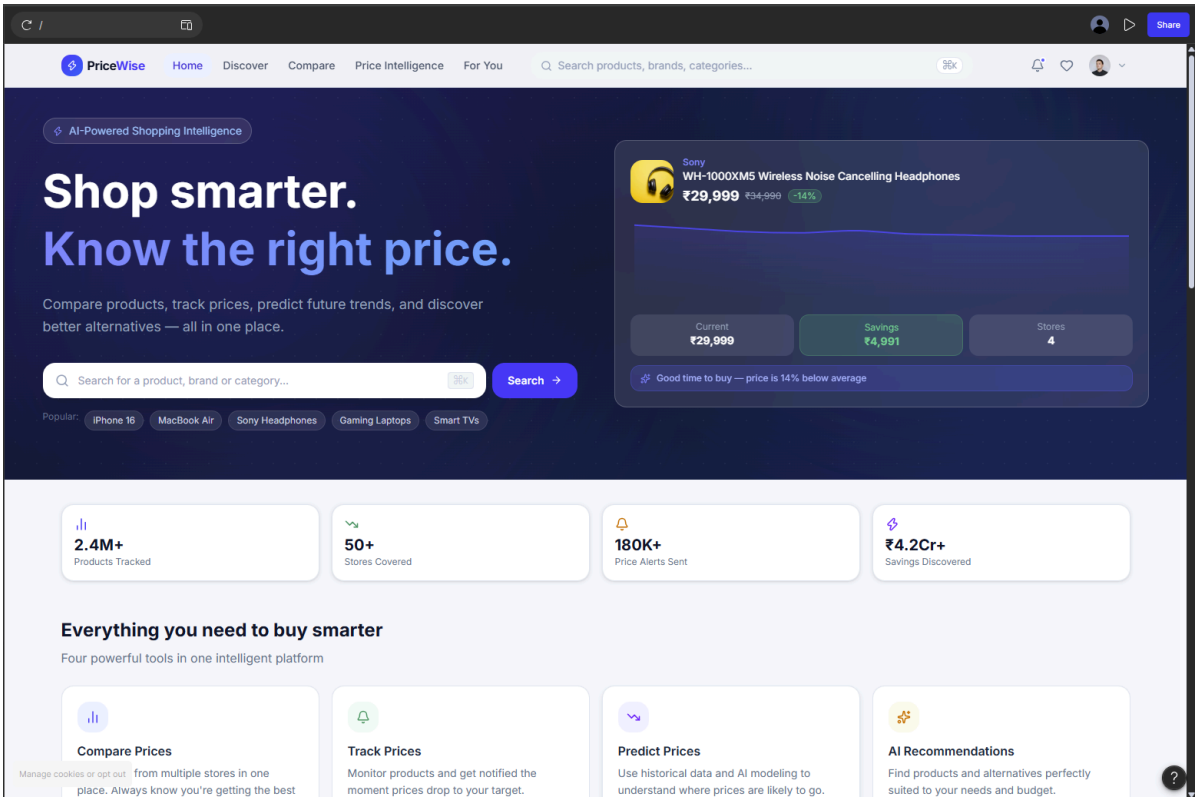


Figure 6.1(b): High-Fidelity Figma UI Mockup — Landing Page & Smart Home Dashboard

C. Interface Component & Layout Breakdown:

UI Component / Region	Visual Design & Functional Specification
Global Navigation Bar	Site logo placeholder, 5 main section navigation links with active state indicator, global search input field with Cmd+K shortcut, and right-aligned user actions (Alerts, Wishlist, Profile).
Hero Search Section	Platform tagline and value proposition headline, explanatory subtitle paragraph, primary search input field with 'Search' CTA button, and quick-filter category pills.
Live Interactive Preview Card	Featured product card containing crossed-box image placeholder, model title, current lowest price vs original MRP, discount badge, 90-day price sparkline trend graph, and AI buy/wait recommendation pill.
Platform Trust Metric Cards	Row of 4 macro statistic cards: Total Products Tracked, Partner Stores Covered, Total Alerts Sent, and Cumulative Savings Discovered.
Core Feature Value Grid	4-column feature pillar cards: Price Comparison, Real-Time Price Tracking, Predictive Price Forecasting, and Personalized AI Recommendations.

D. Screen-Specific Acceptance Criteria:

- Global search input must auto-focus upon clicking Cmd+K shortcut or clicking within the field.
- Trending category pills must redirect to `/discover` with query parameters pre-filtered.
- Featured preview card must dynamically render lowest price available across all crawled retail merchants.
- All primary navigation links must route correctly with active indicator underline on current page.

User Authentication & Sign-In Page

Route: /login (Authentication)

Screen Purpose & User Goal: Provides a clean, frictionless split-screen login and onboarding experience that re-emphasizes key platform benefits while enabling email/password sign-in and Google Single Sign-On (SSO).

A. Low-Fidelity Blueprint Wireframe (Structural Layout & Placeholders):

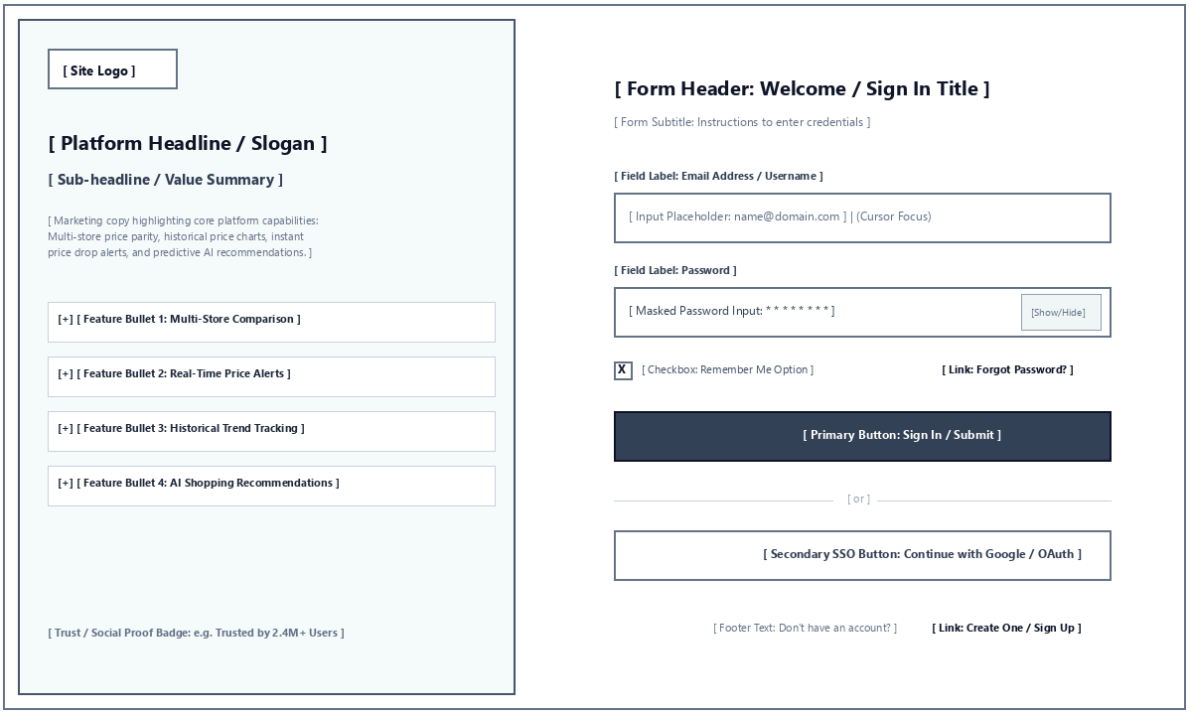


Figure 6.2(a): Low-Fidelity Blueprint Wireframe — User Authentication & Sign-In Page

B. High-Fidelity UI/UX Figma Mockup :

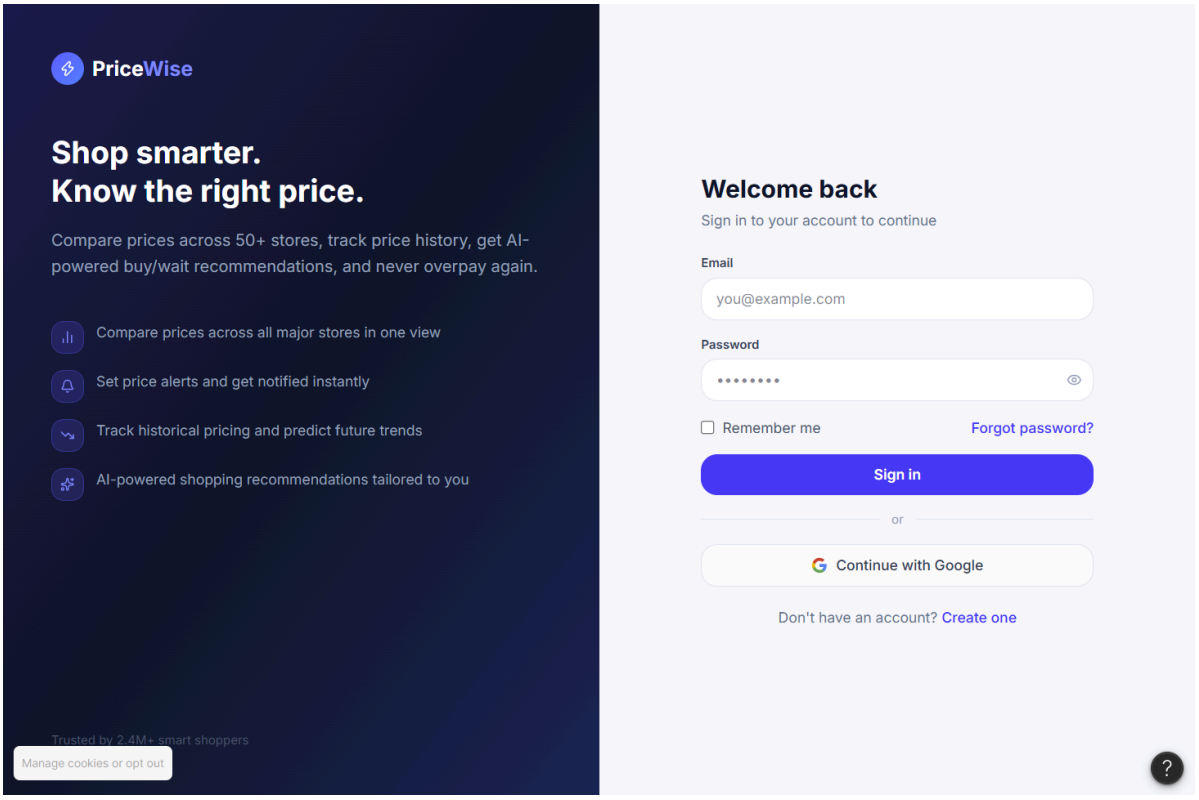


Figure 6.2(b): High-Fidelity Figma UI Mockup — User Authentication & Sign-In Page

C. Interface Component & Layout Breakdown:

UI Component / Region	Visual Design & Functional Specification
Left Value-Proposition Panel	Site logo box, prominent headline, marketing summary text, 4 feature benefit bullet blocks with icons, and social proof trust badge.
Right Authentication Form	Form title ('Welcome back'), instructional subtitle, email address text input field, password text input field with show/hide password toggle button.
Form Controls & Links	'Remember me' persistent session checkbox, 'Forgot password?' recovery text link.
Action CTAs	Primary 'Sign In' submission button, 'or' divider line, and branded 'Continue with Google' OAuth SSO button.
Registration Redirect	Footer link 'Don't have an account? Create one' routing to registration flow.

D. Screen-Specific Acceptance Criteria:

- Form validates email syntax and enforces minimum password length prior to submission.
- Password show/hide toggle switches input field type between masked and visible text.
- Google SSO initiates OAuth 2.0 authentication flow and redirects user to dashboard upon success.
- 'Remember me' checkbox persists session token in encrypted storage.

Product Discovery & Catalog Hub

Route: /discover (Explore Products)

Screen Purpose & User Goal: Enables users to explore multi-category catalogs, view trending products, filter by discount percentages, inspect best-store pricing tags, and receive instant AI buy/wait signals before adding to comparison or tracking.

A. Low-Fidelity Blueprint Wireframe (Structural Layout & Placeholders):

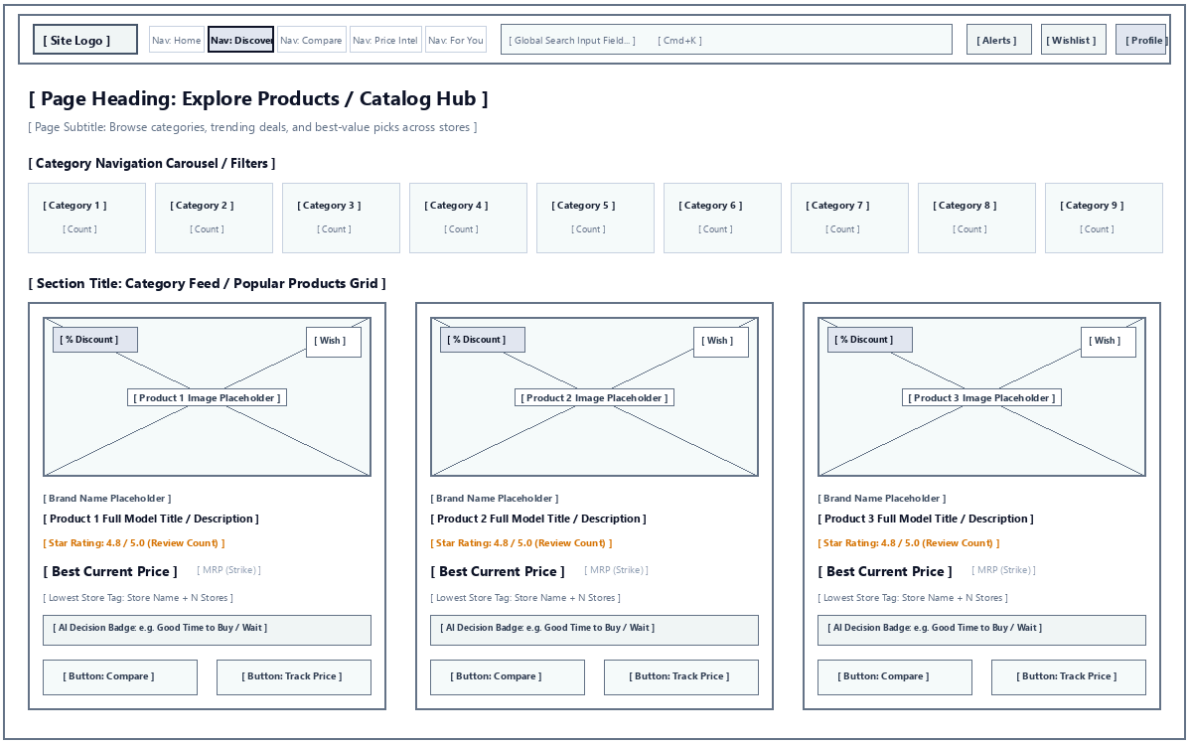


Figure 6.3(a): Low-Fidelity Blueprint Wireframe — Product Discovery & Catalog Hub

B. High-Fidelity UI/UX Figma Mockup :

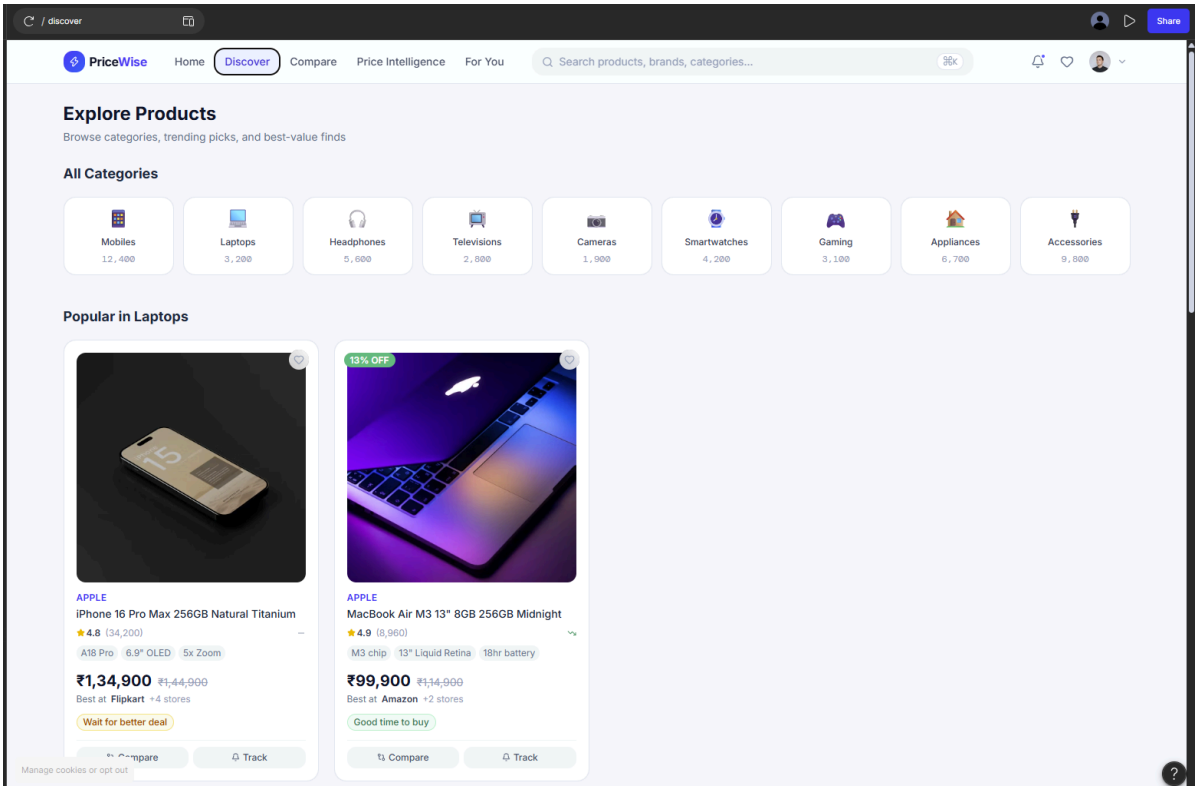


Figure 6.3(b): High-Fidelity Figma UI Mockup — Product Discovery & Catalog Hub

C. Interface Component & Layout Breakdown:

UI Component / Region	Visual Design & Functional Specification
Category Navigation Carousel	Horizontal interactive category pills with item count badges (Mobiles, Laptops, Audio, TVs, Cameras, Smartwatches, Gaming, Appliances, Accessories).
Product Cards Grid	3-column responsive product card grid containing crossed-box image placeholders, percentage discount tag, favorite/wishlist button, brand label, product model title, star rating, and hardware spec tags.
Price & Merchant Badges	Current lowest price, original MRP with strikethrough, lowest store merchant tag (e.g. Best at Flipkart +4 stores), and AI recommendation status badge ('Good time to buy' vs 'Wait for better deal').
Card Action Buttons	Dual CTA buttons: 'Compare' (adds item to comparison tray) and 'Track Price' (opens target price dialog).

D. Screen-Specific Acceptance Criteria:

- Clicking any category tile instantly filters product catalog without requiring a page reload.
- Hovering over product cards triggers subtle elevation and image zoom effects.
- Clicking 'Compare' button increments comparison tray badge count in the navigation bar.
- Clicking 'Track' opens modal dialog to configure custom price drop alerts for that product.

Side-by-Side Product Comparison Matrix

Route: /compare (Product Comparison)

Screen Purpose & User Goal: Offers an in-depth, multi-column comparison table allowing buyers to evaluate 2 to 4 competing products simultaneously across pricing, merchant availability, user sentiment ratings, and technical specifications.

A. Low-Fidelity Blueprint Wireframe (Structural Layout & Placeholders):

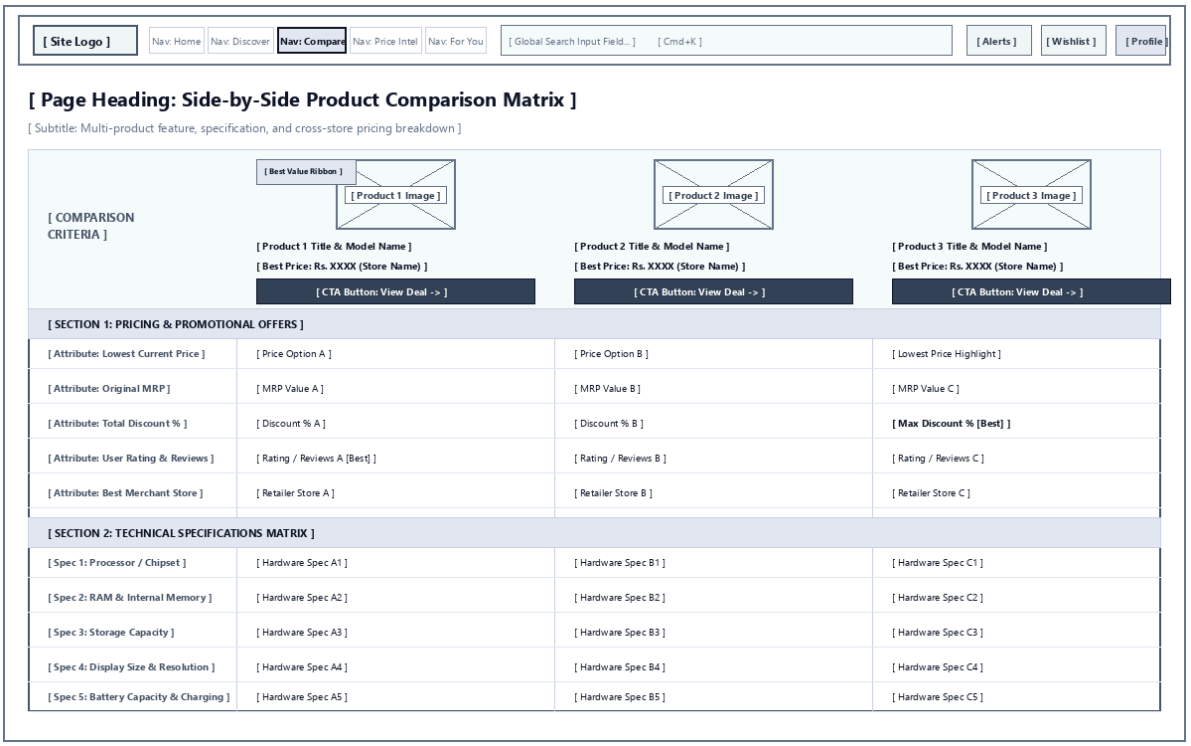


Figure 6.4(a): Low-Fidelity Blueprint Wireframe — Side-by-Side Product Comparison Matrix

B. High-Fidelity UI/UX Figma Mockup :

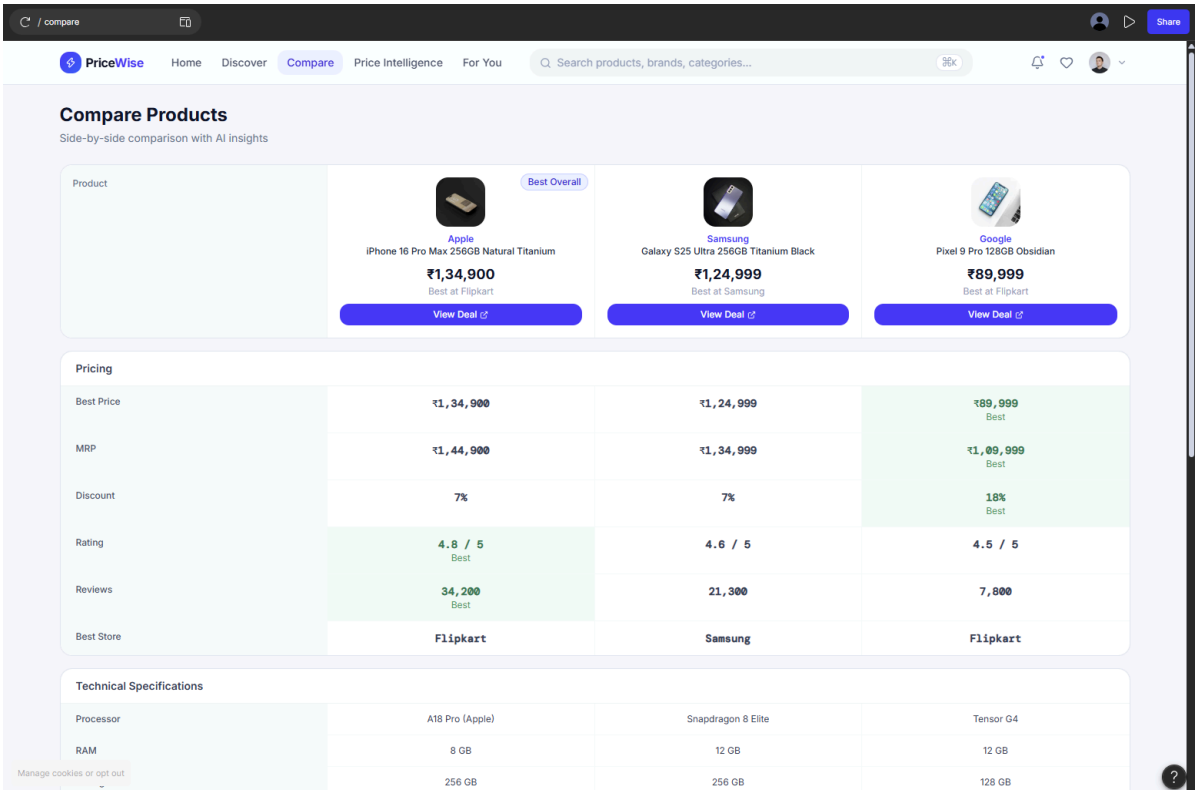


Figure 6.4(b): High-Fidelity Figma UI Mockup — Side-by-Side Product Comparison Matrix

C. Interface Component & Layout Breakdown:

UI Component / Region	Visual Design & Functional Specification
Product Header Row	Crossed-box product image thumbnails, 'Best Value' ribbon badge, product model titles, current best price tags, and primary 'View Deal' external link buttons.
Pricing Breakdown Section	Comparative attribute rows for Best Price, Original MRP, Percentage Discount, User Rating, Review Counts, and Best Store Merchant.
Technical Specifications Matrix	Row-by-row hardware specification breakdown comparing Processors, RAM Memory, Internal Storage, Display Size & Tech, and Battery / Charging specs across all selected items.

D. Screen-Specific Acceptance Criteria:

- Matrix automatically highlights winning attributes in soft green shading with a 'Best' pill badge.
- 'View Deal' CTA buttons open the retailer's verified product page in a new secure browser tab.
- Users can dynamically remove or add additional products to the comparison table.
- Table header row remains sticky when scrolling vertically down long specification sheets.

Price Intelligence & Predictive Analytics Dashboard

Route: /price-intelligence (Analytics & History)

Screen Purpose & User Goal: Delivers advanced price analytics, historical trendlines, market volatility metrics, record lows, and AI-powered 7/30/60-day price forecasting to ensure users never buy right before a major price drop.

A. Low-Fidelity Blueprint Wireframe (Structural Layout & Placeholders):

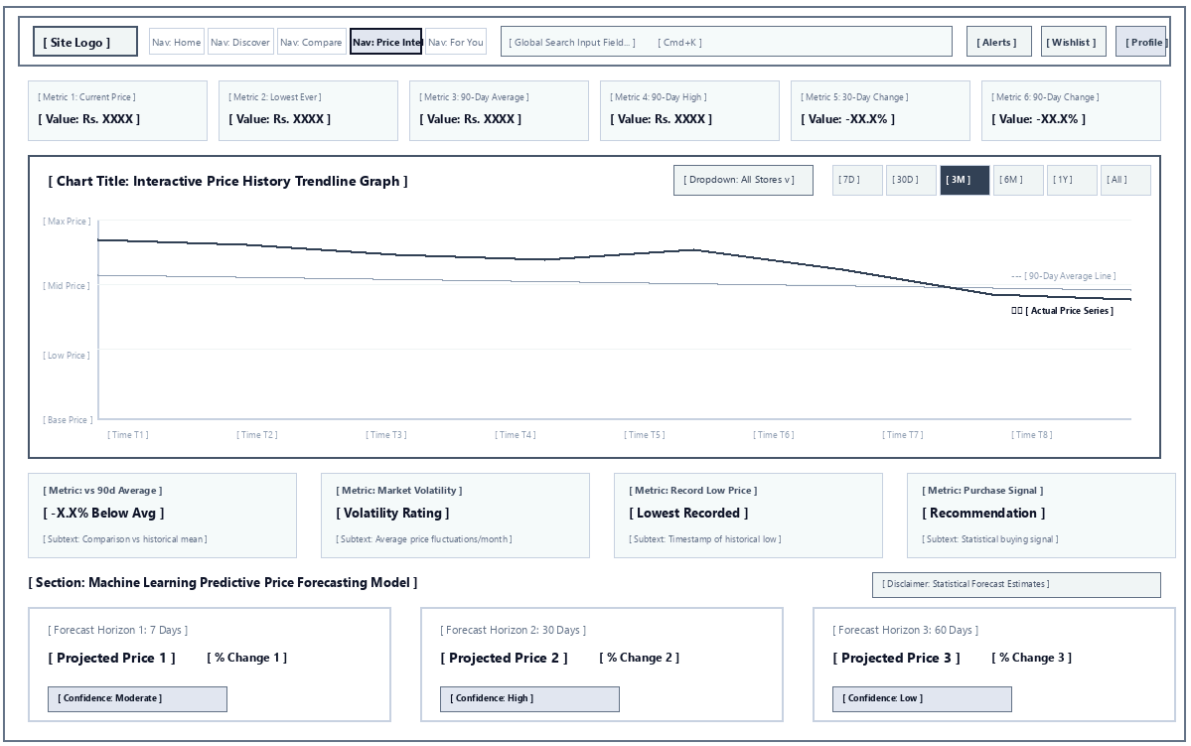


Figure 6.5(a): Low-Fidelity Blueprint Wireframe — Price Intelligence & Predictive Analytics Dashboard

B. High-Fidelity UI/UX Figma Mockup :

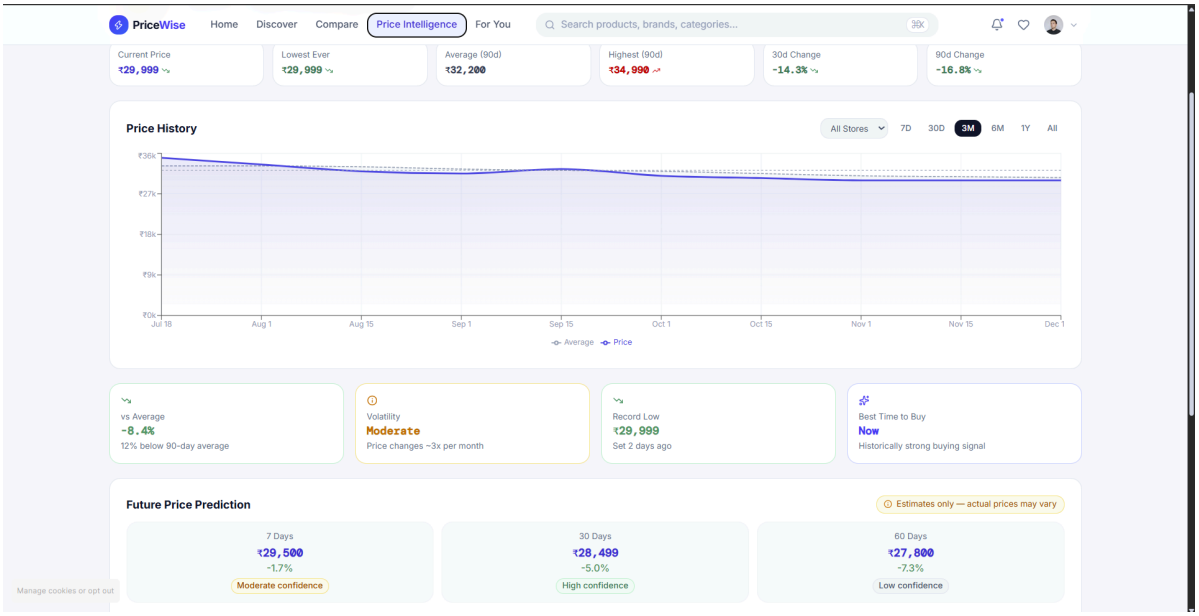


Figure 6.5(b): High-Fidelity Figma UI Mockup — Price Intelligence & Predictive Analytics Dashboard

C. Interface Component & Layout Breakdown:

UI Component / Region	Visual Design & Functional Specification
Top Key Metrics Ribbon	6 summary stat tiles: Current Price, Lowest Ever Recorded, 90-Day Average, 90-Day High, 30-Day Net Change, and 90-Day Net Change.
Interactive Price History Chart	Coordinate axis time-series line chart with dual data series (Actual Price vs 90-Day Average line), store filter dropdown, and timeframe filter pill buttons (7D, 30D, 3M, 6M, 1Y, All).
Market Dynamics Summary Cards	4 analytic insight cards: vs 90-Day Average percentage, Market Volatility score, Record Low timestamp, and Best Time to Buy indicator.
Predictive Price Forecast Horizon	3 machine learning forecast cards projecting 7-Day, 30-Day, and 60-Day price trends with confidence scoring tags and disclaimer note.

D. Screen-Specific Acceptance Criteria:

- Toggling timeframe filters smoothly updates chart coordinate axes and plotted points.
- Hovering over data points displays an interactive tooltip with exact date, store name, and price.
- Confidence badges dynamically adapt color based on certainty (Green = High, Amber = Moderate, Gray = Low).
- Disclaimer banner clearly informs users that future predictions are statistical estimates.

Personalized AI Recommendations ('For You')

Route: /for-you (Personalized Recommendations)

Screen Purpose & User Goal: A personalized product discovery feed that analyzes user interaction history, wishlist entries, and search patterns to recommend tailored deals with explicit rationale banners.

A. Low-Fidelity Blueprint Wireframe (Structural Layout & Placeholders):

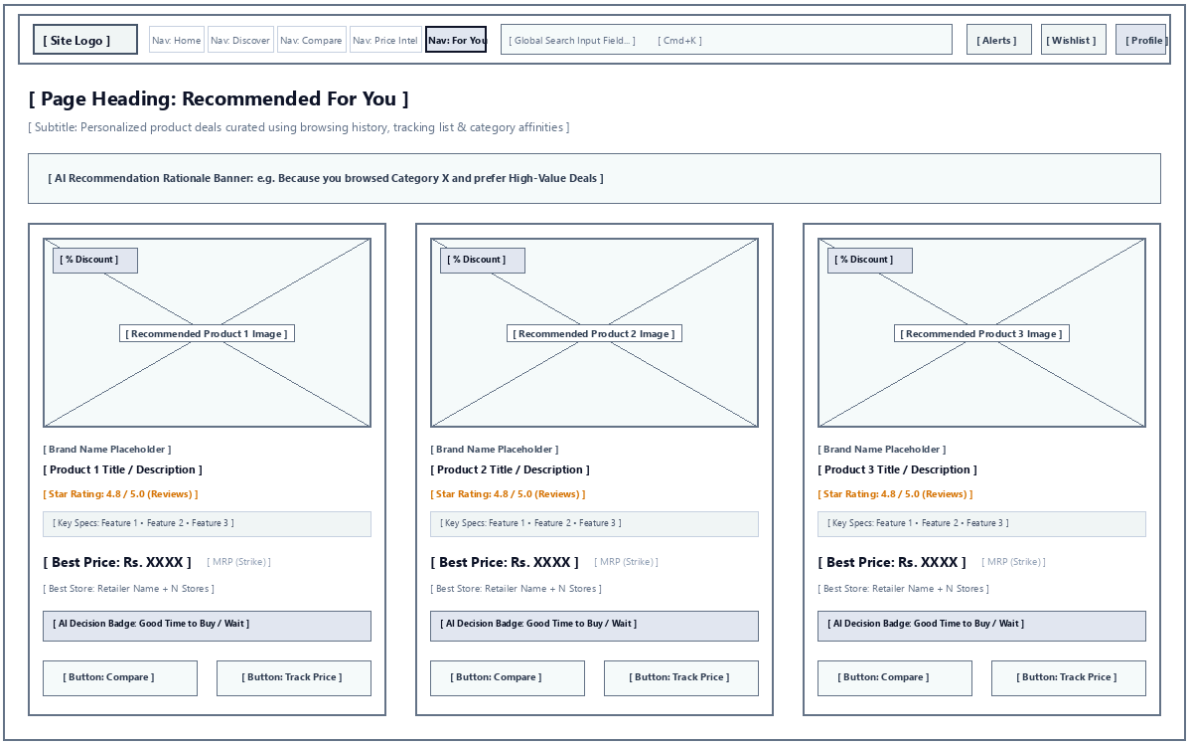


Figure 6.6(a): Low-Fidelity Blueprint Wireframe — Personalized AI Recommendations ('For You')

B. High-Fidelity UI/UX Figma Mockup :

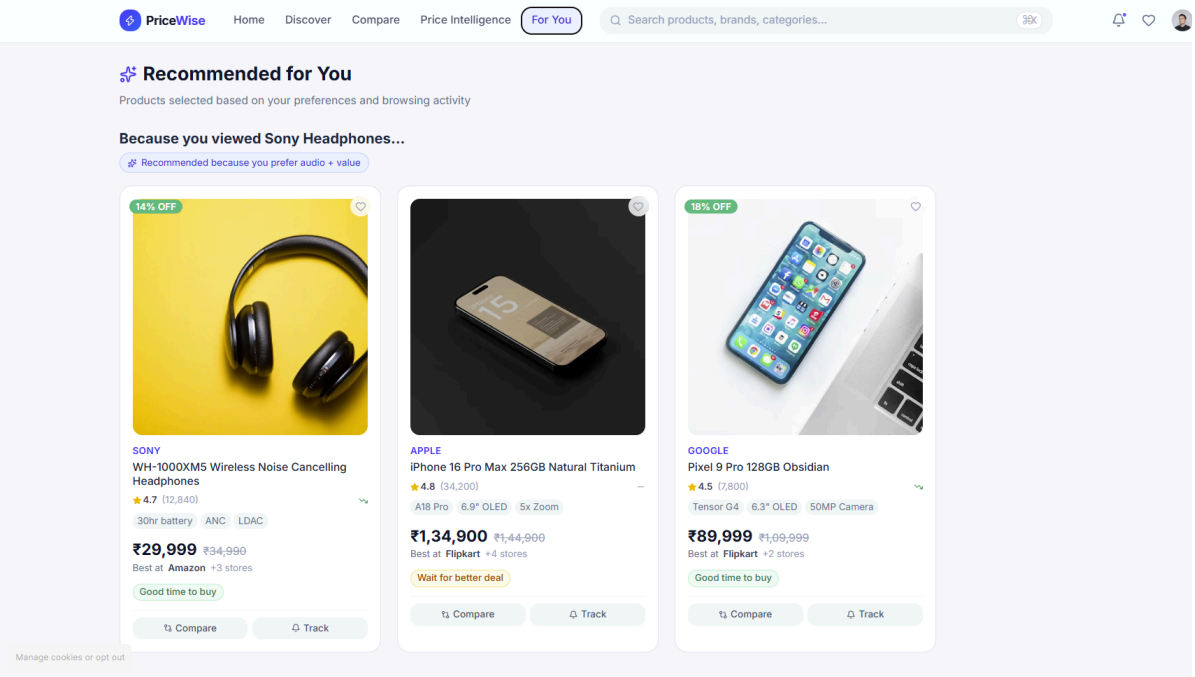


Figure 6.6(b): High-Fidelity Figma UI Mockup — Personalized AI Recommendations ('For You')

C. Interface Component & Layout Breakdown:

UI Component / Region	Visual Design & Functional Specification
Header & Rationale Banner	Section title ('Recommended for You'), descriptive subtitle, and AI recommendation context banner explaining why products were selected (e.g. 'Because you viewed Audio Category...').
Curated Product Cards	Product cards with crossed-box image placeholders, discount badge, brand tag, title, rating, key hardware spec tags, best price vs MRP, best store tag, and AI decision badge.
Quick Action CTAs	Secondary action buttons for 'Compare' and 'Track Price' on each recommended item.

D. Screen-Specific Acceptance Criteria:

- Recommendations refresh dynamically as user browses, tracks, or favorites different categories.
- Rationale banner accurately reflects user's category browsing affinities and price sensitivity.
- Users can dismiss irrelevant recommendations to train the personalization engine.

Tracked Products & Watchlist Management

Route: /watchlist (Price Tracking)

Screen Purpose & User Goal: Enables users to manage active price tracking targets, monitor price gaps in real-time, view visual progress toward desired thresholds, and modify or pause alert triggers

.

A. Low-Fidelity Blueprint Wireframe (Structural Layout & Placeholders):

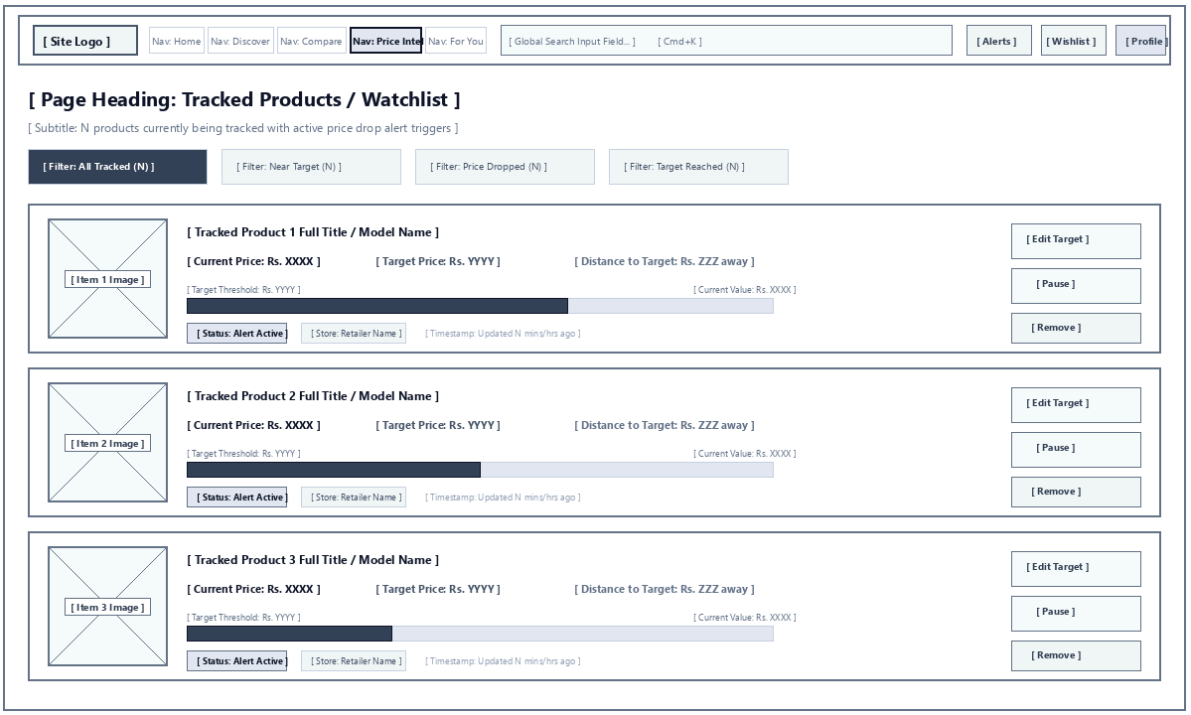


Figure 6.7(a): Low-Fidelity Blueprint Wireframe — Tracked Products & Watchlist Management

B. High-Fidelity UI/UX Figma Mockup :

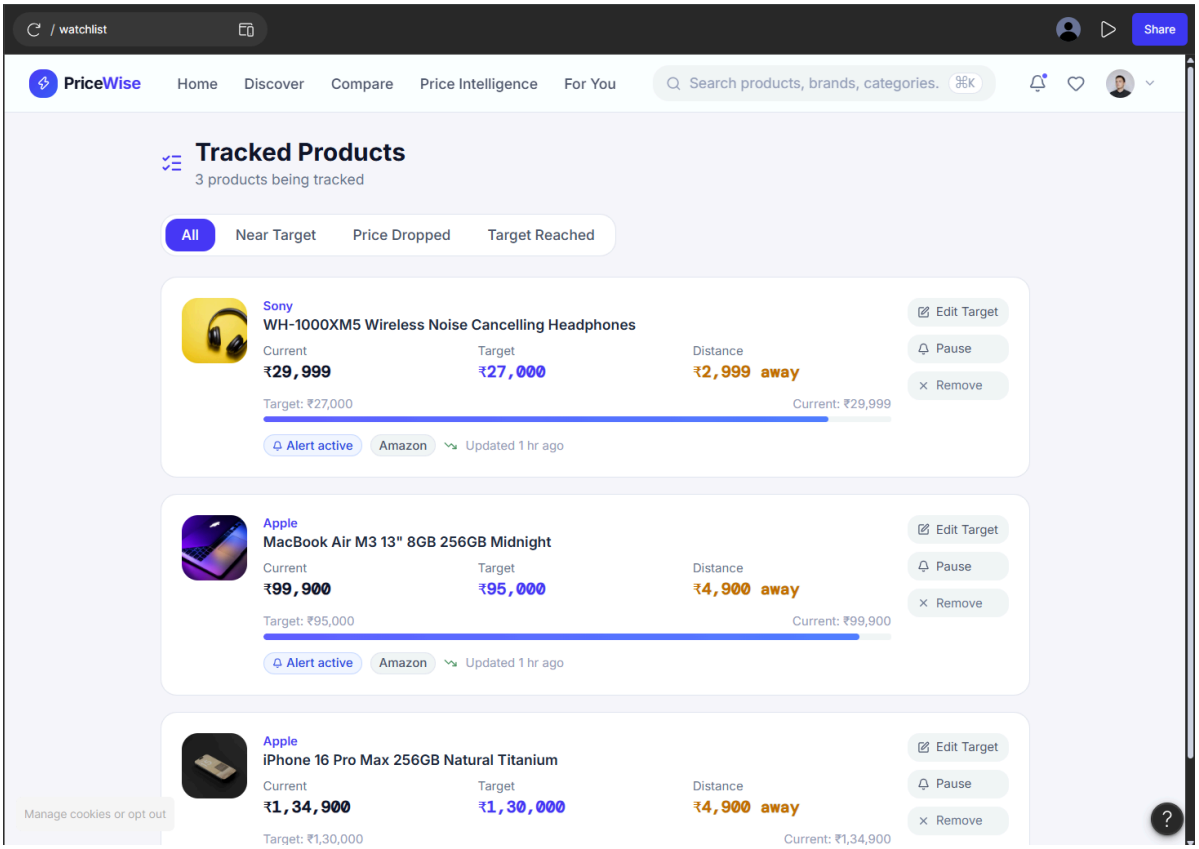


Figure 6.7(b): High-Fidelity Figma UI Mockup — Tracked Products & Watchlist Management

C. Interface Component & Layout Breakdown:

UI Component / Region	Visual Design & Functional Specification
Header & Filter Tabs	Tracked count header ('Tracked Products - N items') and status filter tabs (All, Near Target, Price Dropped, Target Reached).
Tracked Product Card Rows	Horizontal cards with product image placeholder, title, Current Price, Target Price, Distance to Target, and last updated timestamp.
Visual Progress Bar	Horizontal progress bar visualizing current price proximity to user's designated target price threshold.
Alert Status & Actions	Alert active badge, retailer store tag, and right-aligned action buttons: 'Edit Target', 'Pause', and 'Remove'.

D. Screen-Specific Acceptance Criteria:

- Progress bar fills dynamically from left to right as current price approaches designated target price.
- Clicking 'Edit Target' opens an inline input or modal to adjust threshold value.
- Clicking 'Pause' toggles tracking state and updates status badge to 'Alert Paused'.
- Filter tabs accurately filter items based on target proximity and alert status.

User Wishlist & Saved Products

📍 **Route: /wishlist (Saved Items)**

Screen Purpose & User Goal: A consolidated bookmarking interface displaying all saved products, potential cost savings, and historical price movements since the user added them to their wishlist.

A. Low-Fidelity Blueprint Wireframe (Structural Layout & Placeholders):

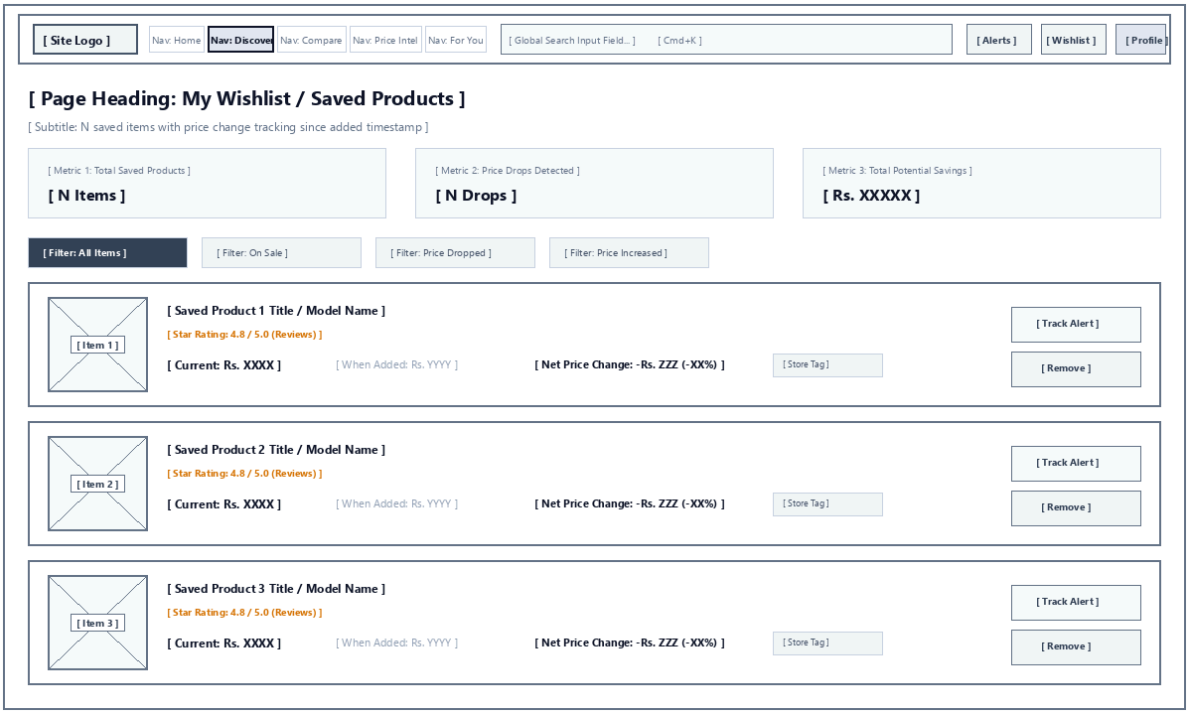


Figure 6.8(a): Low-Fidelity Blueprint Wireframe — User Wishlist & Saved Products

B. High-Fidelity UI/UX Figma Mockup :

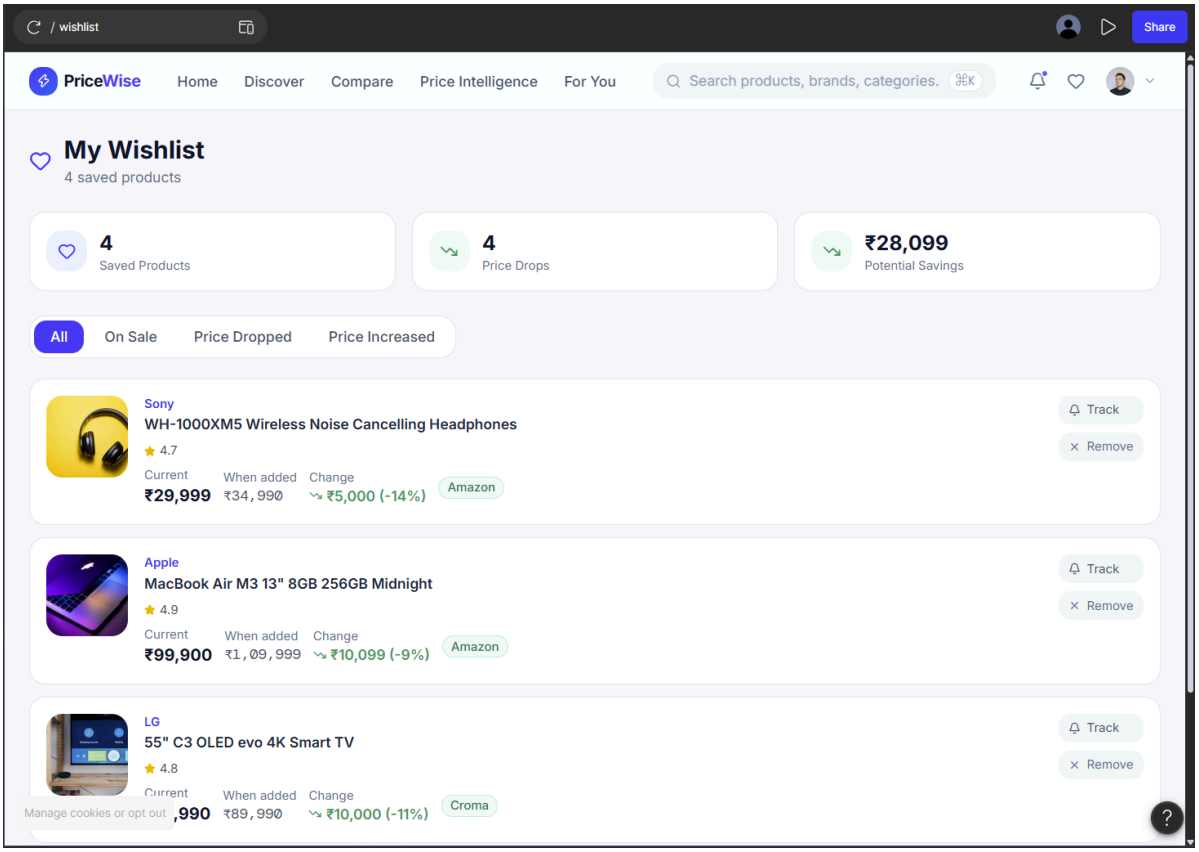


Figure 6.8(b): High-Fidelity Figma UI Mockup — User Wishlist & Saved Products

C. Interface Component & Layout Breakdown:

UI Component / Region	Visual Design & Functional Specification
Wishlist Summary Metrics	3 overview summary tiles: Saved Products Count, Price Drops Detected, and Total Potential Savings.
Filter Bar	Filter buttons for All, On Sale, Price Dropped, and Price Increased.
Saved Product Item Rows	Horizontal item cards with thumbnail image placeholder, title, star rating, Current Price, When Added Price, Net Price Change badge, and store tag.
Row Action CTAs	Secondary action buttons for 'Track' (convert wishlist item to active price alert) and 'Remove' (delete from wishlist).

D. Screen-Specific Acceptance Criteria:

- Total potential savings metric recalculates automatically when items are added or deleted.
- Negative price changes are highlighted in bold green; price increases are highlighted in red.
- Clicking 'Track' converts wishlist item into an active watchlist item with default target price.
- Deleting an item prompts a confirmation toast notification with an 'Undo' option.

Price Drop Alerts & Notification Center

Route: /alerts (Notifications)

Screen Purpose & User Goal: The notification inbox where users review price drop events, target-reached milestones, and direct merchant checkout links for discounted items.

A. Low-Fidelity Blueprint Wireframe (Structural Layout & Placeholders):

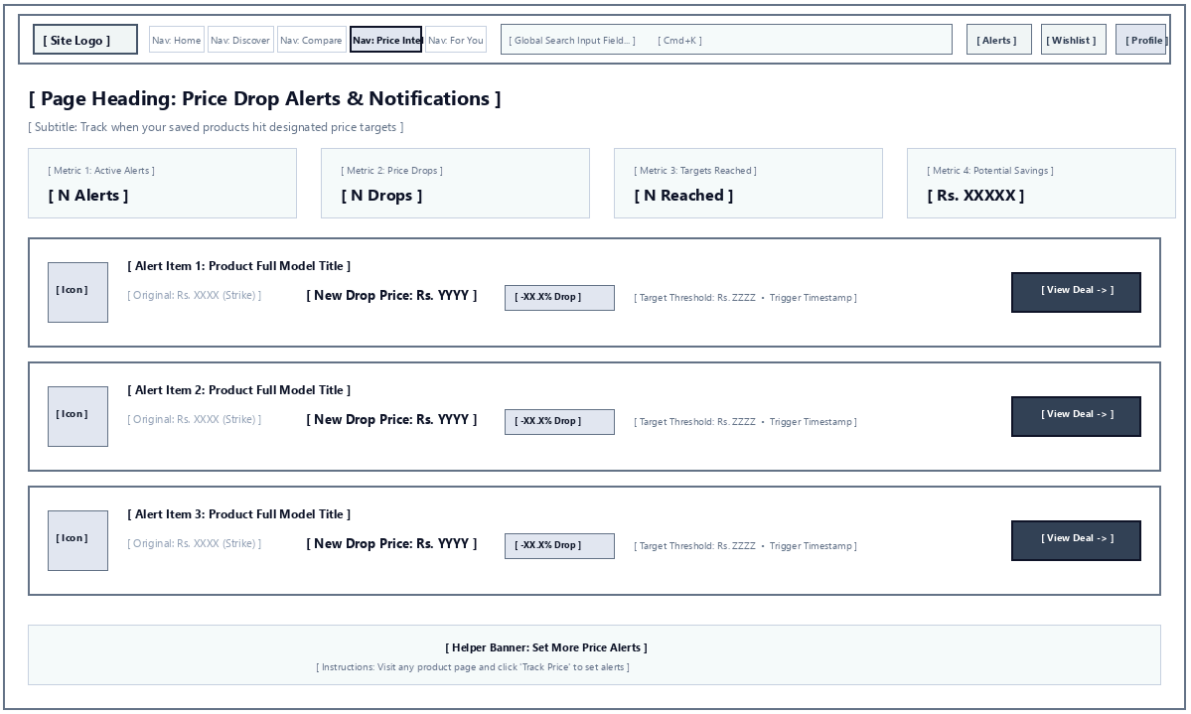


Figure 6.9(a): Low-Fidelity Blueprint Wireframe — Price Drop Alerts & Notification Center

B. High-Fidelity UI/UX Figma Mockup :

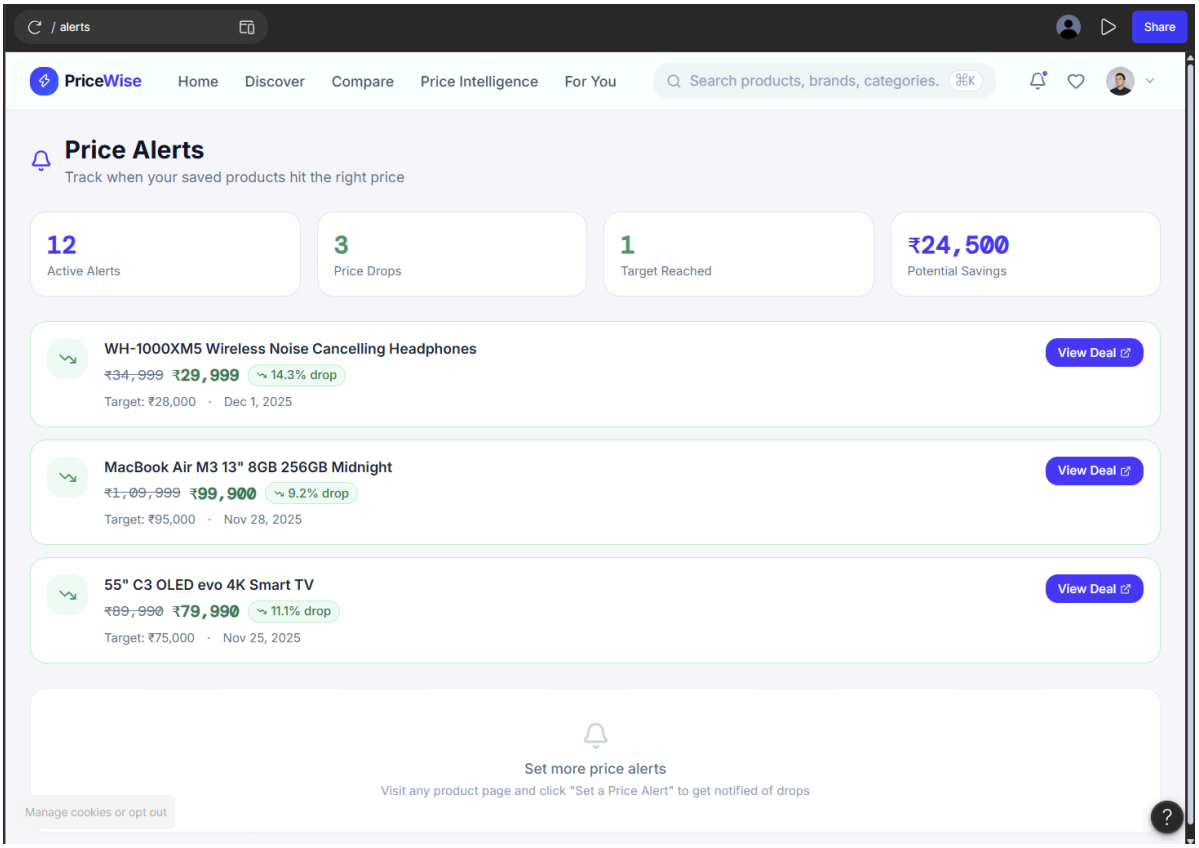


Figure 6.9(b): High-Fidelity Figma UI Mockup — Price Drop Alerts & Notification Center

C. Interface Component & Layout Breakdown:

UI Component / Region	Visual Design & Functional Specification
Alerts Overview Metrics	4 summary metric tiles: Active Alerts, Price Drops, Targets Reached, and Cumulative Potential Savings.
Notification List Rows	Notification cards with price drop icon, product model title, original crossed-out price, new discounted price, percentage drop badge, target price reference, and timestamp.
Direct Checkout CTA	Prominent 'View Deal' button with external link icon opening retailer offer page.
Helper Footer Banner	Subtle helper banner explaining how to configure additional price drop alerts across the catalog.

D. Screen-Specific Acceptance Criteria:

- Unread price drop notifications display an active dot indicator.
- Clicking 'View Deal' routes user through affiliate tracking directly to the retailer product page.
- Notifications can be filtered by 'Price Drops' and 'Targets Reached'.
- Alert timestamps reflect accurate date and time of price drop occurrence.

User Profile & Savings Analytics Hub

Route: /profile (Account & Settings)

Screen Purpose & User Goal: Provides user profile overview, lifetime savings metrics, navigation shortcuts to all platform modules, and quick access to recently viewed products.

A. Low-Fidelity Blueprint Wireframe (Structural Layout & Placeholders):

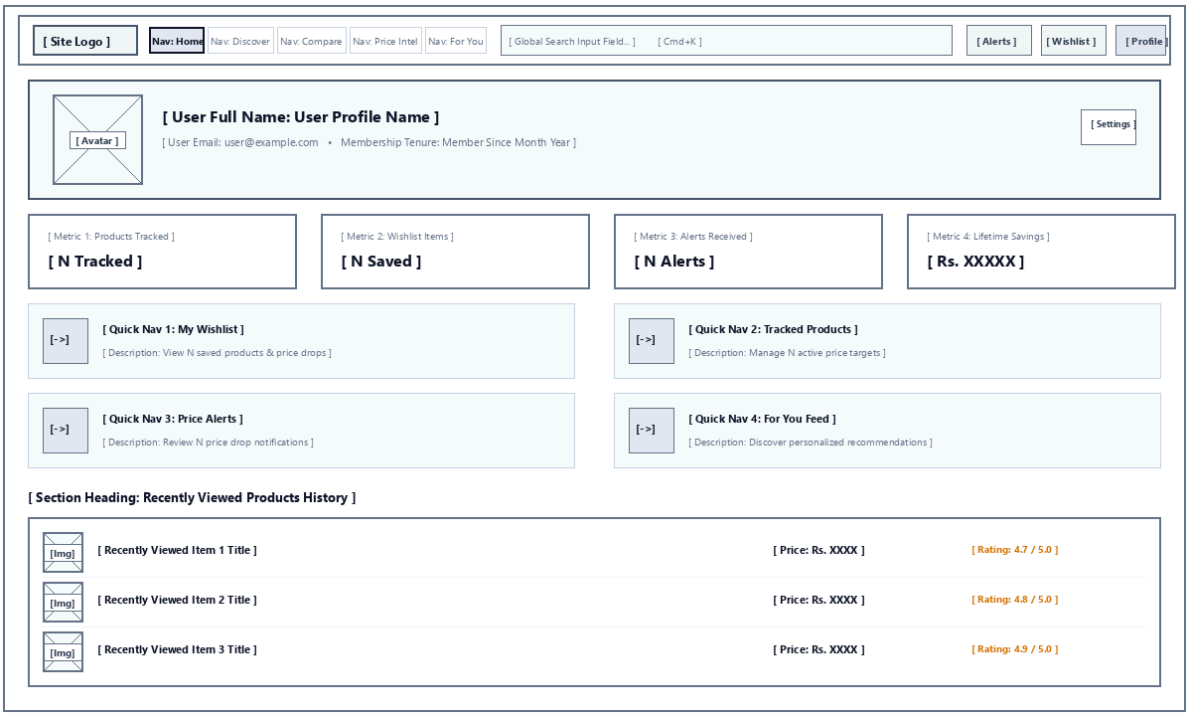


Figure 6.10(a): Low-Fidelity Blueprint Wireframe — User Profile & Savings Analytics Hub

B. High-Fidelity UI/UX Figma Mockup :

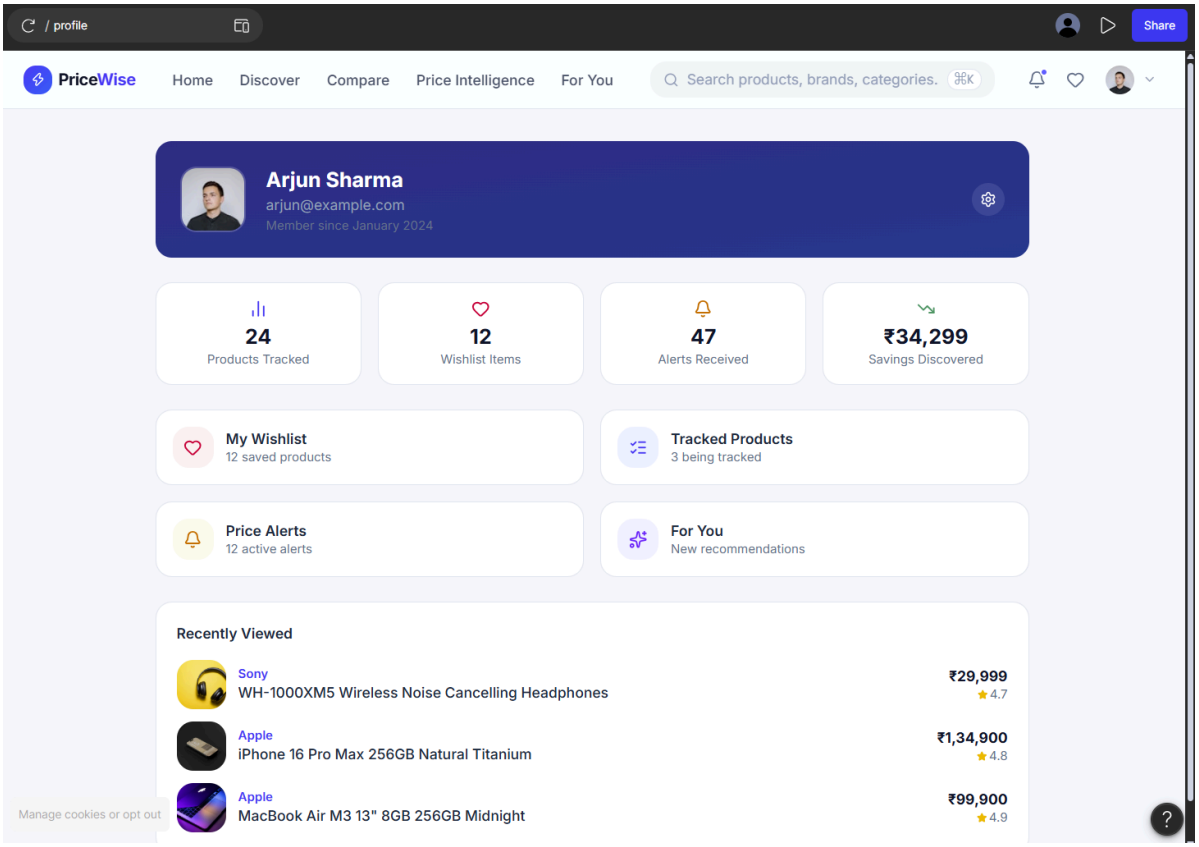


Figure 6.10(b): High-Fidelity Figma UI Mockup — User Profile & Savings Analytics Hub

C. Interface Component & Layout Breakdown:

UI Component / Region	Visual Design & Functional Specification
User Profile Hero Banner	User avatar placeholder, user full name, email address, membership tenure timestamp, and settings configuration button.
Lifetime Key Metrics Grid	4 primary metric cards: Products Tracked, Wishlist Items, Alerts Received, and Total Lifetime Savings Discovered.
Quick Navigation Shortcut Cards	2x2 grid of shortcut cards: My Wishlist, Tracked Products, Price Alerts, and For You Recommendations.
Recently Viewed Products Feed	Compact product history feed showing recently viewed item thumbnails, titles, current prices, and star ratings.

D. Screen-Specific Acceptance Criteria:

- Settings button opens user profile preferences, notification channel toggles, and password management.
- Quick access shortcut cards navigate directly to respective application routes.
- Recently viewed feed maintains a rolling local history of up to 10 most recent items.
- Lifetime savings discovered metric aggregates confirmed discounts below baseline added prices.